Comparative Study of DPU-Based Offloading Approaches

Arjun Ajayakumar[1]
Student
Department of Computer Applications
Mangalam College of Engineering
ajayanarjun608@gmail.com

Sreelekshmi K S[2]
Assistant Professor
Department of Computer Applications
Mangalam College of Engineering
sreelekshmi.ks@mangalam.com

***Abstract*—The rapid growth of cloud computing, data-intensive applications, artificial intelligence workloads, and high-speed networking has placed unprecedented pressure on host CPUs in modern data centers. As network bandwidth and storage throughput continue to scale, an increasing fraction of CPU cycles is consumed by networking, storage management, virtualization, security enforcement, and other infrastructure-related processing. This phenomenon significantly limits application performance, scalability, and energy efficiency, creating what is commonly referred to as the infrastructure processing bottleneck. Traditional approaches based on general-purpose CPUs and fixed-function network interface cards are no longer sufficient to meet these demands.**

Data Processing Units (DPUs), also known as SmartNICs, have emerged as a promising architectural solution to address these challenges. By integrating programmable CPUs, hardware accelerators, onboard memory, and high-speed network interfaces directly into the network interface, DPUs enable the offloading of networking, storage, and computation tasks from the host CPU. Unlike conventional accelerators, DPUs operate as independent compute entities capable of executing operating systems and complex software stacks, thereby enabling a new paradigm of heterogeneous computing in data centers. This shift allows host CPUs to focus on application-level computation while DPUs efficiently handle data-path-intensive and infrastructure workloads.

This paper presents a comparative study of three representative research works that explore DPU-based offloading from different perspectives. The first work, referred to as Base0.1, is a comprehensive survey on heterogeneous computing using SmartNICs and emerging DPUs. It provides a broad overview of DPU architectures, programming models, application domains, and open research challenges, establishing a conceptual foundation for understanding the DPU ecosystem. The second work, Base0.2, investigates protocol-level offloading by transparently migrating the entire TCP stack onto off-path DPUs. This approach demonstrates that even complex, stateful, and latency-sensitive protocols can be efficiently executed on programmable DPU cores, resulting in significant reductions in host CPU utilization and improved network performance. The third work, Base0.3, adopts a system-level perspective and proposes a holistic framework for accelerating cloud data processing workloads using DPUs by coordinating compute, network, and storage engines.

The comparison is conducted across multiple dimensions, including scope of contribution, offloading granularity, architectural design, data-path handling, programmability, performance objectives, and limitations. Through this analysis, the paper highlights the trade-offs between fine-grained protocol-level offloading and coarse-grained application-level acceleration. The study demonstrates that these approaches are complementary and collectively illustrate the evolution of DPUs from fixed-function accelerators into fully programmable, heterogeneous compute nodes. Finally, the paper identifies key future research directions, including standardized programming abstractions, portability across DPU vendors, and unified frameworks that seamlessly integrate protocol-level and system-level offloading in next-generation data center architectures.

This paper presents a comparative study of three representative DPU-based research works. Base0.1 serves as the primary reference, providing a survey of heterogeneous computing using DPUs. Base0.2 demonstrates protocol-level offloading through full TCP stack migration to DPUs, while Base0.3 proposes a system-level framework for cloud data processing. The comparison evaluates scope, offloading granularity, architecture, programmability, performance goals, and limitations, highlighting current trends and future directions in DPU-based heterogeneous computing.

***Keywords*—Data Processing Units, SmartNICs, heterogeneous computing, TCP offloading, data processing**

I. INTRODUCTION

Modern data centers are increasingly constrained by CPU bottlenecks arising from networking, storage management, virtualization, and security processing. As cloud services, data-intensive applications, and high-speed networks continue to expand, a growing portion of host CPU cycles is consumed by infrastructure-related tasks rather than application logic. Although network bandwidth and storage throughput have increased significantly over the past decade, improvements in general-purpose CPU performance have not scaled at the same pace. This imbalance has led to inefficiencies commonly referred to as the *infrastructure tax*, which negatively impacts system scalability, energy efficiency, and overall performance in contemporary data centers.

Traditionally, network interface cards (NICs) were designed as simple I/O devices responsible only for packet transmission and reception. To meet increasing performance demands, NICs evolved into programmable SmartNICs capable of offloading selected networking functions such as checksum computation, segmentation, encryption, and virtualization support. Building upon this evolution, Data Processing Units (DPUs) represent a more advanced class of SmartNICs that integrate programmable multi-core CPUs, specialized hardware accelerators, onboard memory, and high-speed networking interfaces

negatively impacts system scalability, energy efficiency, and overall performance in contemporary data centres.

Traditionally, network interface cards (NICs) were designed as simple I/O devices responsible only for packet transmission and reception. To meet increasing performance demands, NICs evolved into programmable SmartNICs capable of offloading selected networking functions such as checksum computation, segmentation, encryption, and virtualization support. Building upon this evolution, Data Processing Units (DPUs) represent a more advanced class of SmartNICs that integrate programmable multi-core CPUs, specialized hardware accelerators, onboard memory, and high-speed networking interface within a single system-on-chip. Unlike fixed-function accelerators, DPUs are capable of executing operating systems and complex software stacks, enabling them to function as independent compute entities alongside host CPUs.

DPUs enable a heterogeneous computing paradigm in which infrastructure and application workloads can be executed independently of the host CPU. By offloading data-path-intensive tasks such as protocol processing, virtual switching, storage virtualization, and security enforcement to DPUs, host CPUs are freed to focus on application-level computation. This separation of responsibilities improves performance isolation, reduces CPU contention, and enhances energy efficiency. As a result, DPUs are increasingly being adopted in large-scale cloud data centres to accelerate critical infrastructure services and improve overall system utilization.

Despite their advantages, effectively exploiting DPUs presents several challenges. Programming DPUs often requires expertise in low-level frameworks and hardware-specific soft- ware development kits, increasing development complexity. Additionally, heterogeneity across DPU vendors, limited on- board resources, and abstraction mismatches between application requirements and low-level hardware interfaces complicate portability and system integration. Designers must also carefully select offloading granularity to balance performance gains against data movement overheads and coordination costs between the host and the DPU.

In response to these challenges, a growing body of research has explored DPU-based computing from multiple perspectives, ranging from high-level surveys to protocol-specific offloading mechanisms and system-level frameworks. This paper presents a comparative study of three major research works in this domain. The first provides a comprehensive survey of heterogeneous computing using DPUs, establishing a conceptual foundation for understanding their capabilities and limitations. The second investigates protocol-level offloading by migrating the entire TCP stack onto DPUs, demonstrating the feasibility of fine-grained infrastructure offloading. The third proposes a system-level framework that integrates DPUs into cloud data processing pipelines. Through this comparison, the paper analyses how DPUs are currently exploited and discusses key directions for the evolution of future DPU-based systems.

DPUs enable heterogeneous computing by executing infrastructure and application workloads independently of the host CPU. This allows CPUs to focus on application logic while DPUs handle data-path-intensive tasks. Despite their advantages, effective DPU utilization introduces challenges related to programmability, abstraction, and system integration. This paper compares three major research works to analyse how DPUs are currently exploited and how future systems may evolve.

II. RELATED WORKS

Early research on SmartNICs primarily focused on accelerating packet processing and network functions in order to reduce host CPU overhead and improve network throughput. Systems such as ClickOS, NetVM, and NetBricks demonstrated that network functions could be executed efficiently in user space or lightweight virtual machines, significantly reducing kernel involvement and enabling high-performance networking on commodity hardware. These works laid the foundation for network function virtualization and highlighted the benefits of moving data-path processing closer to the network interface.

The emergence of programmable data planes further advanced SmartNIC research by enabling flexible and protocol- independent packet processing. The introduction of the P4 programming language allowed developers to define packet parsing and processing logic independent of underlying hard- ware, greatly improving programmability and adaptability. In parallel, the Data Plane Development Kit (DPDK) enabled kernel-bypass networking, allowing applications to achieve high throughput and low latency by directly accessing network devices from user space. These technologies became essential building blocks for many SmartNIC- and DPU-based systems.

More recent research has expanded the role of SmartNICs into fully programmable Data Processing Units (DPUs) ca- pable of executing operating systems, runtime environments, and application-level logic. Unlike earlier SmartNIC designs that focused primarily on networking tasks, DPUs integrate programmable CPUs, hardware accelerators, onboard mem- ory, and high-speed networking interfaces, allowing them to function as independent compute nodes within the datacentre. Major cloud providers have adopted DPUs to offload virtualization, storage, and security functions, improving performance isolation, scalability, and energy efficiency in multi-tenant environments.

In addition to infrastructure offloading, research efforts have explored protocol-level and system-level DPU utilization. Protocol-level approaches, such as full TCP stack offloading, demonstrate that complex, stateful protocols can be executed efficiently on programmable DPU cores, significantly reducing host CPU utilization. System-level frameworks extend DPU usage beyond

networking by integrating DPUs into data processing pipelines, enabling acceleration of compute- and I/O-intensive workloads. Despite these advances, challenges related to programmability, portability across vendors, and lack of standardized abstractions remain unresolved.

In contrast to prior work that typically focuses on a single layer of the system stack, this paper provides a unified comparative analysis of survey-based insights, protocol- level offloading mechanisms, and system-level data processing frameworks. By examining these approaches together, the study highlights key design trade-offs and research directions for future DPU-based heterogeneous computing systems.

III. METHODOLOGY

This paper adopts a qualitative comparative analysis methodology to systematically evaluate three representative research works in the domain of Data Processing Unit (DPU)- based heterogeneous computing. Rather than proposing a new architecture or system implementation, the objective of this study is to analyze existing approaches in order to identify key design choices, trade-offs, and emerging research trends related to DPU utilization in modern data centres.

. Base0.1 is selected as the primary reference for this study due to its comprehensive survey nature and broad coverage of DPU architectures, programming models, application do- mains, and open challenges. The taxonomy, classifications, and research gaps identified in Base0.1 provide the conceptual framework that guides the comparative analysis. By grounding the evaluation in a survey-based reference, this methodology ensures that the comparison is aligned with the current state of the art and reflects widely recognized challenges in the field. Base0.2 and Base0.3 are chosen as representative implementations of two distinct DPU utilization strategies. Base0.2 exemplifies protocol-level offloading by migrating the entire TCP stack onto programmable DPU cores, focusing on reducing host CPU overhead for latency-sensitive networking workloads. In contrast, Base0.3 represents a system-level approach that integrates DPUs into cloud data processing pipelines, enabling the coordinated offloading of compute, network, and storage operations. Together, these works capture different abstraction levels and design philosophies within the DPU research landscape.

The comparison is conducted across multiple evaluation dimensions derived from the challenges and opportunities highlighted in Base0.1. These dimensions include scope and research objectives, offloading granularity, architectural design and data-path handling, programmability and software stack requirements, performance goals, and system limitations. Each paper is first analyzed independently to understand its core contributions and assumptions. Subsequently, a cross-paper comparison is performed to identify similarities, differences, and complementary aspects among the approaches.

This structured methodology ensures a fair, consistent, and unbiased evaluation of the selected works. By focusing on qualitative analysis rather than direct performance bench-marking, the methodology emphasizes conceptual clarity and design insight. The outcome of this process is a holistic understanding of how DPUs are currently exploited at different layers of the computing stack and how future systems may integrate protocol-level and system-level offloading within unified, portable, and scalable DPU-based frameworks.

IV. COMPARATIVE ANALYSIS

This section presents a detailed comparative analysis of the three selected research works, with Base0.1 serving as the primary reference. The analysis highlights how the design goals, architectural choices, and abstraction levels of Base0.2 and Base0.3 align with and extend the opportunities identified in the survey presented in Base0.1.

A. Scope and Research Objective

Among the three works, Base0.1 has the broadest scope, providing a comprehensive survey of the entire DPU ecosystem. It examines architectural trends, programming models, application domains, and open challenges associated with DPUs, thereby establishing a conceptual foundation for heterogeneous computing. In contrast, Base0.2 focuses on a narrow but critical infrastructure component—TCP protocol processing—aiming to reduce host CPU overhead through complete protocol offloading. Base0.3 targets cloud data processing workloads and proposes a system-level framework that exploits DPUs to accelerate compute-, network-, and storage- intensive operations. From the perspective of Base0.1, both Base0.2 and Base0.3 can be viewed as practical realizations of the offloading opportunities identified in the survey.

B. Offloading Granularity

Base0.1 discusses offloading at multiple levels of the system stack, ranging from low-level infrastructure services to application-level workloads. This multi-level perspective highlights the trade-offs between fine-grained control and system complexity. Base0.2 adopts a fine-grained approach by mi- grating the entire TCP protocol stack onto the DPU, enabling precise control over networking behavior and minimizing host CPU involvement. In contrast, Base0.3 operates at a higher abstraction level by offloading data processing operators and I/O pipelines rather than individual protocols. This coarse-grained approach simplifies integration with existing systems while trading off some low-level control.

C. Architecture and Data Path

Base0.1 distinguishes between on-path and off-path Smart-NIC architectures and emphasizes the flexibility of off-path designs. Both Base0.2 and Base0.3 adopt off-path DPU architectures, allowing DPUs to function as independent compute entities. Base0.2 optimizes latency-sensitive TCP data paths by minimizing PCIe interactions and batching

DMA transfers between the host and the DPU. In contrast, Base0.3 focuses on throughput-oriented designs by constructing asynchronous and pipelined data paths that span compute, network, and storage engines, prioritizing scalability over ultra-low latency.

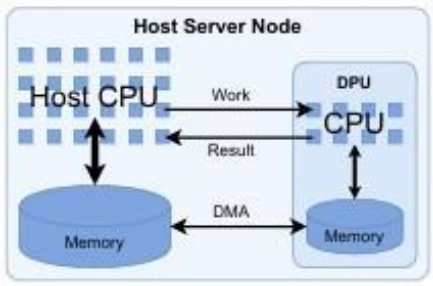

Fig. 1 Architecture

D. Programmability

Programmability is a central concern highlighted in Base0.1. Base0.2 relies heavily on DPDK-based custom TCP stacks, which achieve high performance but require deep expertise in low-level networking and DPU programming. This approach can limit portability and developer accessibility. Base0.3 ad- dresses these challenges by providing higher-level abstractions and unified APIs that hide vendor-specific details, enabling more portable and developer-friendly DPU utilization. This design directly responds to the programmability and abstraction challenges identified in Base0.1.

E. Limitations

Each approach exhibits distinct limitations. Base0.1 identifies fundamental challenges such as lack of standardization, limited DPU resources, and programming complexity. Base0.2 is highly specialized for networking workloads and does not generalize easily to other application domains. Base0.3, while more flexible, introduces additional complexity in scheduling, coordination, and resource management across multiple engines. These limitations highlight the trade-offs between specialization and generality in DPU-based system design.

F. Summary Comparison

Overall, the comparative analysis reveals that protocol-level and system-level offloading approaches are complementary rather than competing. Base0.2 demonstrates the effectiveness of fine-grained protocol offloading, while Base0.3 illustrates the benefits of holistic, system-level DPU integration. Base0.1 provides the unifying perspective that connects these approaches and outlines key directions for future research in heterogeneous computing with DPUs.

V. EXPERIMENTAL SETUP

This study is based on experimental evaluations reported in the selected base papers rather than on independent implementation or benchmarking. The objective of this section is to summarize the experimental environments, workloads, and evaluation methodologies employed in the referenced works in order to provide context for the comparative analysis presented in this paper. By relying on previously validated experiments, the study ensures that conclusions are grounded in realistic system behaviour and representative data centre workloads.

Base0.2 evaluates protocol-level offloading by implementing a fully offloaded TCP stack on off-path Data Processing Units, specifically NVIDIA BlueField DPUs. The experimental setup includes hosts equipped with high-speed network interfaces connected to DPUs featuring programmable ARM cores and onboard memory. The evaluation uses real-world, widely adopted applications such as Redis, Lighttpd, and HAProxy to measure the impact of TCP offloading on host CPU utilization, throughput, and latency. Workloads include a mix of small-packet and latency-sensitive traffic patterns, allowing the study to assess the effectiveness of offloading un- der realistic operating conditions. Performance metrics focus on CPU savings, network throughput, and end-to-end latency, providing a comprehensive view of protocol-level offloading benefits between the host and the DPU. The evaluation focuses on data-intensive operations such as compression, encryption, predicate pushdown, aggregation, and input/output processing. These workloads are designed to stress both compute and I/O subsystems, highlighting the benefits of coordinated offloading across compute, network, and storage engines. Metrics include application throughput, resource utilization, and cost efficiency, reflecting performance characteristics relevant to large-scale cloud environments. By analyzing experimentally validated results from both protocol-level and system-level evaluations, this paper avoids the confounding factors associated with synthetic benchmarks or simplified micro-tests. Instead, it relies on realistic work- loads and production-relevant configurations to draw meaningful conclusions. This experimental context enables a fair comparison of different offloading strategies and supports the broader objective of understanding how DPUs can be effectively integrated into modern heterogeneous data center architectures.

VI. RESULTS AND DISCUSSION

The experimental results reported in the selected base papers demonstrate that Data Processing Units (DPUs) can significantly reduce host CPU utilization while maintaining or improving overall system performance. Across both protocol- level and system-level evaluations, DPUs effectively offload data-path-intensive tasks, validating their role as key enablers of heterogeneous computing in modern data centers. Base0.2 reports substantial reductions in host CPU utilization by fully offloading the TCP protocol stack onto off- path DPUs. By executing all TCP-related

functionality on programmable DPU cores, the system minimizes host CPU involvement in connection management, congestion control, retransmissions, and packet processing. Experimental results show that this approach is particularly effective for latency- sensitive and small-packet workloads, where TCP processing overhead traditionally dominates CPU usage. In several cases, throughput is maintained or improved while achieving significant CPU savings, demonstrating that fine-grained protocol- level offloading can deliver both performance and efficiency benefits. These results confirm the feasibility of executing complex, stateful protocols on programmable DPUs without relying on fixed-function hardware accelerators.

In contrast, Base0.3 focuses on system-level offloading for data-intensive workloads and reports improvements in both throughput and cost efficiency. By coordinating compute, network, and storage engines on the DPU, Base0.3 accelerates operations such as compression, encryption, predicate push- down, and I/O processing. The reported results indicate that offloading these operations reduces data movement between the host and storage systems, leading to better resource utilization and improved end-to-end performance. Unlike protocol- level offloading, which targets specific infrastructure functions, the system-level approach demonstrates broader applicability across diverse workloads commonly found in cloud data processing environments.

Comparing the results of Base0.2 and Base0.3 reveals an important trade-off between specialization and flexibility. Protocol-level offloading, as exemplified by Base0.2, achieves high efficiency and low latency for well-defined networking tasks but is tightly coupled to specific protocols and work- loads. System-level frameworks, as demonstrated by Base0.3, offer greater flexibility and portability across applications but introduce additional complexity in scheduling, coordination, and resource management. These trade-offs closely align with the observations and challenges synthesized in Base0.1, which emphasizes the need to balance fine-grained control with system-wide abstraction. Overall, the results reinforce the central insight of Base0.1 that DPUs are most effective when treated as programmable, heterogeneous compute nodes rather than simple accelerators. The complementary nature of protocol-level and system-level offloading suggests that future DPU-based systems should integrate both approaches, enabling DPUs to support latency- sensitive infrastructure services alongside throughput-oriented data processing workloads within unified architecture.

VII. CONCLUSION AND FUTURE SCOPE

This paper presented a detailed comparative study of three representative research works that explore Data Processing Unit (DPU)-based heterogeneous computing, with Base0.1 serving as the primary reference. By examining comprehensive surveys, a protocol-level offloading system, and a system- level data processing framework, the study analyzed how DPUs are currently leveraged at different layers of the computing stack to address the growing performance, scalability, and efficiency challenges faced by modern data centers. The comparative analysis highlights not only the diversity of existing DPU-based solutions but also the fundamental architectural and design trade-offs involved in exploiting programmable, network-attached computing resources. Base0.1 establishes a foundational understanding of the DPU ecosystem by surveying architectural trends, programming models, application domains, and open research challenges. It positions DPUs as programmable, heterogeneous compute nodes rather than fixed-function accelerators, emphasizing their potential to fundamentally reshape data center architectures. By categorizing existing work across infrastructure offloading, benchmarking, and application-level acceleration, Base0.1 provides a unifying framework that helps contextualize subsequent system designs. Importantly, the survey identifies critical challenges such as limited onboard resources, programming complexity, abstraction mismatch, and lack of standardization, which continue to influence the design of DPU-based systems.

Building on this foundation, Base0.2 demonstrates that fine-grained, protocol-level offloading is both feasible and effective. By migrating the entire TCP stack onto off- path DPUs, Base0.2 shows that even complex, stateful, and latency-sensitive protocols can be efficiently executed on programmable DPU cores. The reported results indicate significant reductions in host CPU utilization while maintaining or improving network throughput and latency, particularly for small-packet and latency-sensitive workloads. This work pro- vides concrete validation of the infrastructure offloading opportunities identified in Base0.1 and challenges the traditional assumption that such protocols must reside on the host CPU or be implemented using fixed-function hardware. However, the specialization of this approach also highlights the trade-off between achieving maximum efficiency for specific workloads and maintaining generality across diverse applications. In contrast, Base0.3 illustrates how DPUs can be integrated into higher-level software systems through a holistic, system-oriented framework that targets cloud data processing workloads. By coordinating compute, network, and storage engines on the DPU, Base0.3 demonstrates how DPUs can accelerate a wide range of data-intensive operations, including compression, encryption, predicate pushdown, and I/O processing. This system-level approach aligns closely with the broader offloading opportunities identified in Base0.1 and highlights the role of DPUs in improving both performance and cost efficiency in cloud environments. The emphasis on abstraction and portability in Base0.3 directly addresses some of the programmability and heterogeneity challenges identified in the survey, although it introduces additional complexity in scheduling, coordination, and runtime management.

The comparative analysis conducted in this paper shows that protocol-level and system-level offloading approaches are complementary rather than competing. Fine-grained protocol offloading, as exemplified by Base0.2, delivers high efficiency and low latency for well-defined infrastructure functions but is typically specialized and tightly coupled to specific workloads. System-level frameworks, as

demonstrated by Base0.3, offer greater flexibility and broader applicability across diverse applications but require more sophisticated runtime systems and abstraction layers. Together, these approaches illustrate the breadth of the DPU design space and reinforce the central insight of Base0.1: that DPUs should be viewed as general- purpose heterogeneous computer resources capable of supporting multiple classes of workloads. From a broader architectural perspective, the findings of this study suggest that the future of DPU-based computing lies in the convergence of these approaches. Rather than choosing between protocol-level and application-level offloading, next- generation systems are likely to integrate both strategies within a unified architecture. Such systems would enable DPUs to simultaneously support latency-sensitive infrastructure services, such as networking and security, alongside throughput- oriented data processing workloads. Achieving this integration will require advances in programming abstractions, runtime coordination, and system-level resource management.

Looking ahead, several important research directions emerge from this study. First, the development of standardized and portable programming models is essential to reduce development complexity and improve cross-vendor compatibility. The lack of standardization remains a major barrier to widespread DPU adoption, as developers are often required to rely on vendor-specific SDKs and low-level frameworks.

REFERENCES

[1] [1] N. Tibbetts *et al.*, "A survey on heterogeneous computing using SmartNICs and emerging data processing units," *arXiv preprint*, 2025.

[2] [2] H. Nan *et al.*, "Plug & offload: Transparently offloading TCP stack onto off-path SmartNIC," *arXiv preprint*, 2025.

[3] [3] J. Hu *et al.*, "DPDPU: Data processing with DPUs," *arXiv preprint*, 2024.

[4] [4] M. Firestone *et al.*, "Azure accelerated networking: SmartNICs in the public cloud," in *Proc. USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2018.

[5] [5] A. Panda *et al.*, "NetBricks: Taking the V out of NFV," in *Proc. USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2016.

[6] [6] P. Bosshart *et al.*, "P4: Programming protocol-independent packet processors," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 44, no. 3, pp. 87–95, 2014.

[7] [7] Intel Corporation, "Data Plane Development Kit (DPDK)," White Paper, 2023.

[8] [8] NVIDIA Corporation, "BlueField data processing unit architecture," Technical White Paper, 2023.

[9] [9] J. Martins *et al.*, "ClickOS and the art of network function virtualization," in *Proc. USENIX NSDI*, 2014.

[10] [10] E. Kohler *et al.*, "The Click modular router," *ACM Trans. Comput. Syst.*, vol. 18, no. 3, pp. 263–297, 2000.

[11] [11] S. Novakovic *et al.*, "Application performance in the cloud: The need for SmartNICs," *IEEE Computer*, vol. 51, no. 11, pp. 36–44, 2018.

[12] [12] A. Kaufmann *et al.*, "FlexNIC: Rethinking network DMA," in *Proc. USENIX OSDI*, 2018.

[13] [13] M. Alizadeh *et al.*, "Data center TCP (DCTCP)," in *Proc. ACM SIGCOMM Conf.*, 2010.

[14] [14] K. Tan *et al.*, "SENIC: Scalable NIC for end-host rate limiting," in *Proc. USENIX NSDI*, 2015.

[15] [15] R. Mittal *et al.*, "TIMELY: RTT-based congestion control for the datacenter," in *Proc. ACM SIGCOMM Conf.*, 2015.

[16] [16] J. Sherry *et al.*, "Making middleboxes someone else's problem," in *Proc. ACM SIGCOMM Conf.*, 2012.

[17] [17] C. Kim *et al.*, "NetVM: High performance and flexible networking using virtualization on commodity platforms," *IEEE Trans. Comput.*, vol. 63, no. 1, pp. 41–57, 2014.

[18] [18] Y. Li *et al.*, "ClickNP: Highly flexible and high-performance network processing with reconfigurable hardware," in *Proc. ACM SIGCOMM Conf.*, 2016.

[19] [19] S. Han *et al.*, "Network support for resource disaggregation in next-generation datacenters," in *Proc. ACM HotNets Workshop*, 2015.

[20] [20] M. Zaharia *et al.*, "Apache Spark: A unified engine for big data processing," *Commun. ACM*, vol. 59, no. 11, pp. 56–65, 2016.